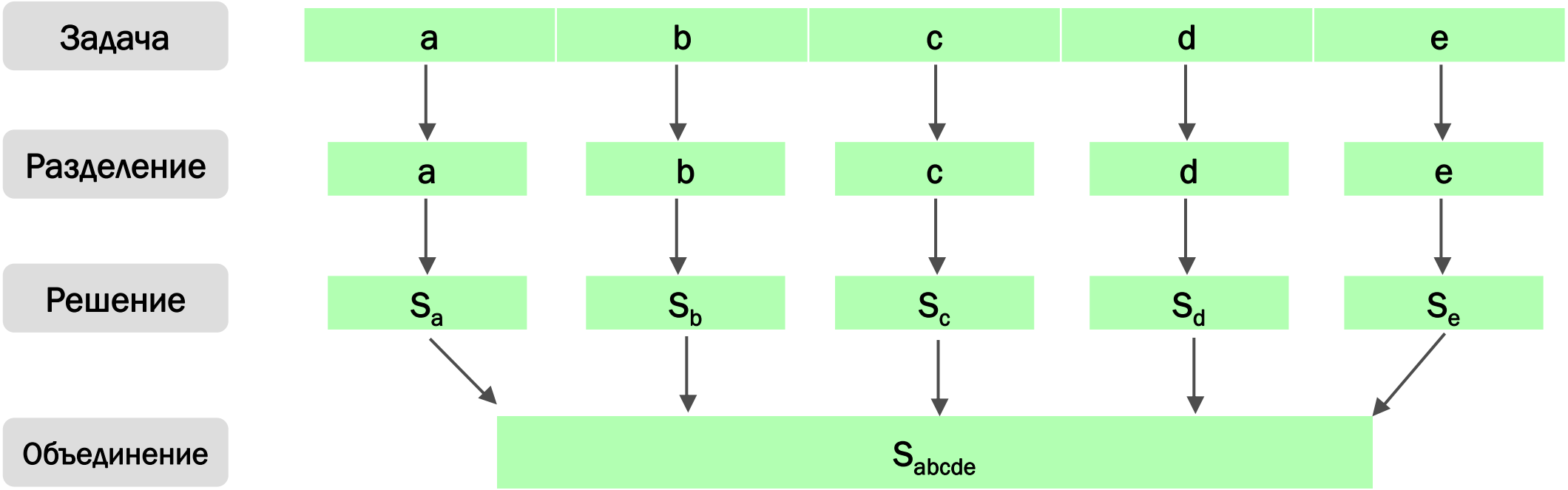
МЕТОД РАЗРАБОТКИ АЛГОРИТМОВ. РАЗДЕЛЯЙ И ВЛАСТВУЙ

Буров Арсений 211 гр.

Разделяй и властвуй - это метод разработки алгоритмов, основанный на рекурсивном разбиении сложной задачи на две или более подзадачи того же типа, но меньшего размера, до тех пор, пока они не станут элементарными и решаемыми напрямую.



Этапы работы

1. Разделение исходной задачи на независимые подзадачи меньшей размерности;
2. Рекурсивное решение каждой подзадачи; если подзадача достаточно мала, решаем её непосредственно (базовый случай рекурсии);
3. Объединение решений подзадач в решение исходной задачи.

Примеры алгоритмов

- **Сортировка слиянием** - делит массив пополам, рекурсивно сортирует каждую половину и сливает два отсортированных массива в один.
- **Быстрая сортировка** - выбирает опорный элемент, разделяет массив на элементы меньше и больше опорного, затем рекурсивно сортирует части.
- **Бинарный поиск** - алгоритм, который последовательно делит пополам заранее отсортированный массив данных, чтобы обнаружить нужный элемент.
- **Алгоритм Карацубы** - умножение больших чисел за $O(n^{\log_2(3)})$ вместо классического $O(n^2)$ за счёт разбиения на части и уменьшения количества перемножений.
- **Алгоритм Штрассена** – алгоритм быстрого умножения квадратных матриц, является обобщением метода умножения Карацубы на матрицы.

Математически работа алгоритмов “разделяй и властвуй” описывается с помощью рекуррентного соотношения. Чаще всего используется формула, которая легла в основу Основной теоремы о рекуррентных соотношениях:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- $T(n)$ — общее время выполнения алгоритма для задачи размером n.
- a — количество подзадач, на которые разбивается исходная задача ($a \geq 1$).
- $\frac{n}{b}$ — размер каждой подзадачи (предполагается, что все они примерно равны). Параметр $b > 1$ показывает, во сколько раз уменьшается объем данных.
- $f(n)$ — стоимость работы по разделению задачи и комбинированию результатов (всё, что происходит вне рекурсивных вызовов).

Пример: Алгоритм Карацубы

Два n-битовых числа можно представить в виде $ax + b$, $cx + d$, где $x = 2^{\lceil n/2 \rceil}$ и $a, b, c, d < 2^{\lceil n/2 \rceil}$.

Умножение на $2^{\lceil n/2 \rceil}$ и сложение делаются за время $O(1)$. Поэтому задача умножения сводится к вычислению коэффициентов многочлена $(ax + b)(cx + d)$. Используя тождество:

$$(a + b)(c + d) = ac + (ad + bc) + bd,$$

этот многочлен можно представить в виде:

$$(ax + b)(cx + d) = ac\,x^2 + (ad + bc)\,x + bd.$$

Подставляя выражение для средней суммы коэффициентов, получаем:

$$(ax + b)(cx + d) = ac\,x^2 + \big((a + b)(c + d) - ac - bd\big)x + bd.$$

Рассмотрим пример умножения двух восьмизначных (в десятичной записи) чисел $N = 11113333$ и $M = 55557777$.

Каждое из них представляется в виде суммы двух чисел вдвое меньшей разрядности, одно из которых взято со сдвигом:

$$\begin{cases} N = 1111 \cdot 10000 + 3333 \\ M = 5555 \cdot 10000 + 7777 \end{cases}$$

Раскрывая скобки, произведение N и M можно переписать как:

$$(1111 \cdot 10000 + 3333)(5555 \cdot 10000 + 7777) =$$

$$= 1111 \cdot 5555 \cdot 10000^2 + 10000(1111 \cdot 7777 + 5555 \cdot 3333) + 3333 \cdot 7777$$

Карацуба нашёл, что вместо четырёх умножений вдвое более коротких чисел, можно делать лишь три: $1111 \cdot 5555$, $(1111 + 3333) \cdot (5555 + 7777)$ и $3333 \cdot 7777$, в результате чего выражение преобразуется к виду:

$$1111 \cdot 5555 \cdot 10000^2 + 10000\big((1111 + 3333) \cdot (5555 + 7777) -$$

$$-1111 \cdot 5555 - 3333 \cdot 7777\big) + 3333 \cdot 7777$$

Для умножения укороченных вдвое чисел применяется тот же алгоритм: $1111 \cdot 5555$ представляется как $(11 \cdot 100 + 11)(55 \cdot 100 + 55)$ и так далее. На практике для достаточно коротких чисел применяется уже обычное умножение как более эффективное.

Оценка сложности

$$\boxed{T(n) = 3T\left(\frac{n}{2}\right) + cn,}$$

Раскроем по шагам:

$$T(n) = 3T\left(\frac{n}{2}\right) + cn.$$

$$T\left(\frac{n}{2}\right) = 3T\left(\frac{n}{4}\right) + c\frac{n}{2},$$

поэтому

$$T(n) = 3\left[3T\left(\frac{n}{4}\right) + c\frac{n}{2}\right] + cn = 3^2T\left(\frac{n}{4}\right) + 3c\frac{n}{2} + cn.$$

Продолжая, через k шагов (пока $\frac{n}{2^k} = 1$, т.е. $k = \log_2 n$):

$$T(n) = 3^kT(1) + cn\left(1 + \frac{3}{2} + \left(\frac{3}{2}\right)^2 + \cdots + \left(\frac{3}{2}\right)^{k-1}\right).$$

Сумма в скобках — геометрическая прогрессия со знаменателем $3/2$:

$$S = \sum_{i=0}^{k-1}\left(\frac{3}{2}\right)^i = \frac{\left(\frac{3}{2}\right)^k - 1}{\frac{3}{2} - 1} = 2\left(\left(\frac{3}{2}\right)^k - 1\right).$$

Так как $k = \log_2 n$, то

$$\left(\frac{3}{2}\right)^k = 3^k/2^k = n^{\log_2 3}/n^{\log_2 2} = n^{\log_2 3 - 1}.$$

Тогда

$$T(n) \le 3^kT(1) + 2cn \cdot n^{\log_2 3 - 1} = 3^kT(1) + 2cn^{\log_2 3}.$$

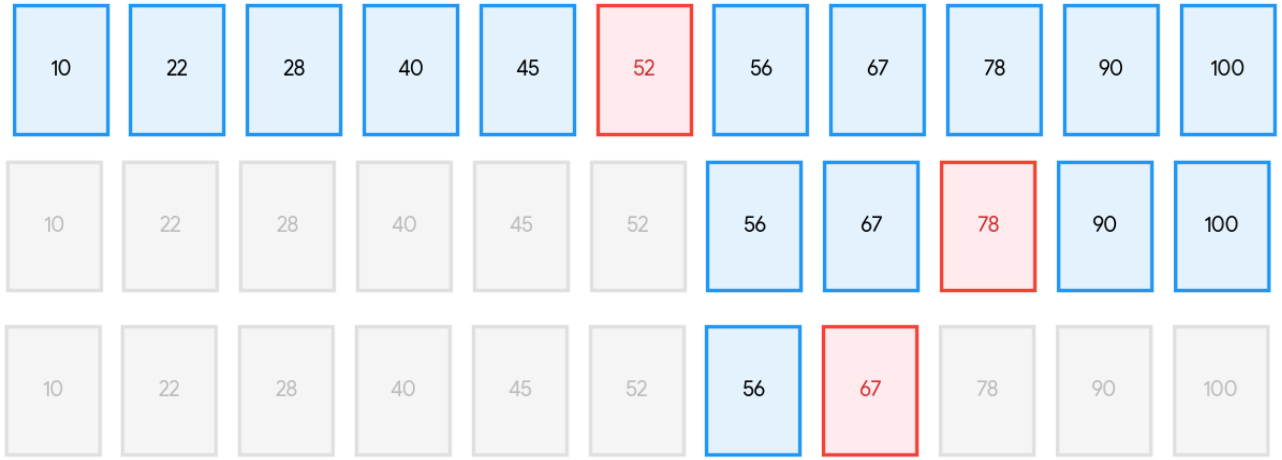
Учитывая, что $3^k = 3^{\log_2 n} = n^{\log_2 3}$, получаем

$$T(n) = O\left(n^{\log_2 3}\right).$$

Пример: Бинарный поиск

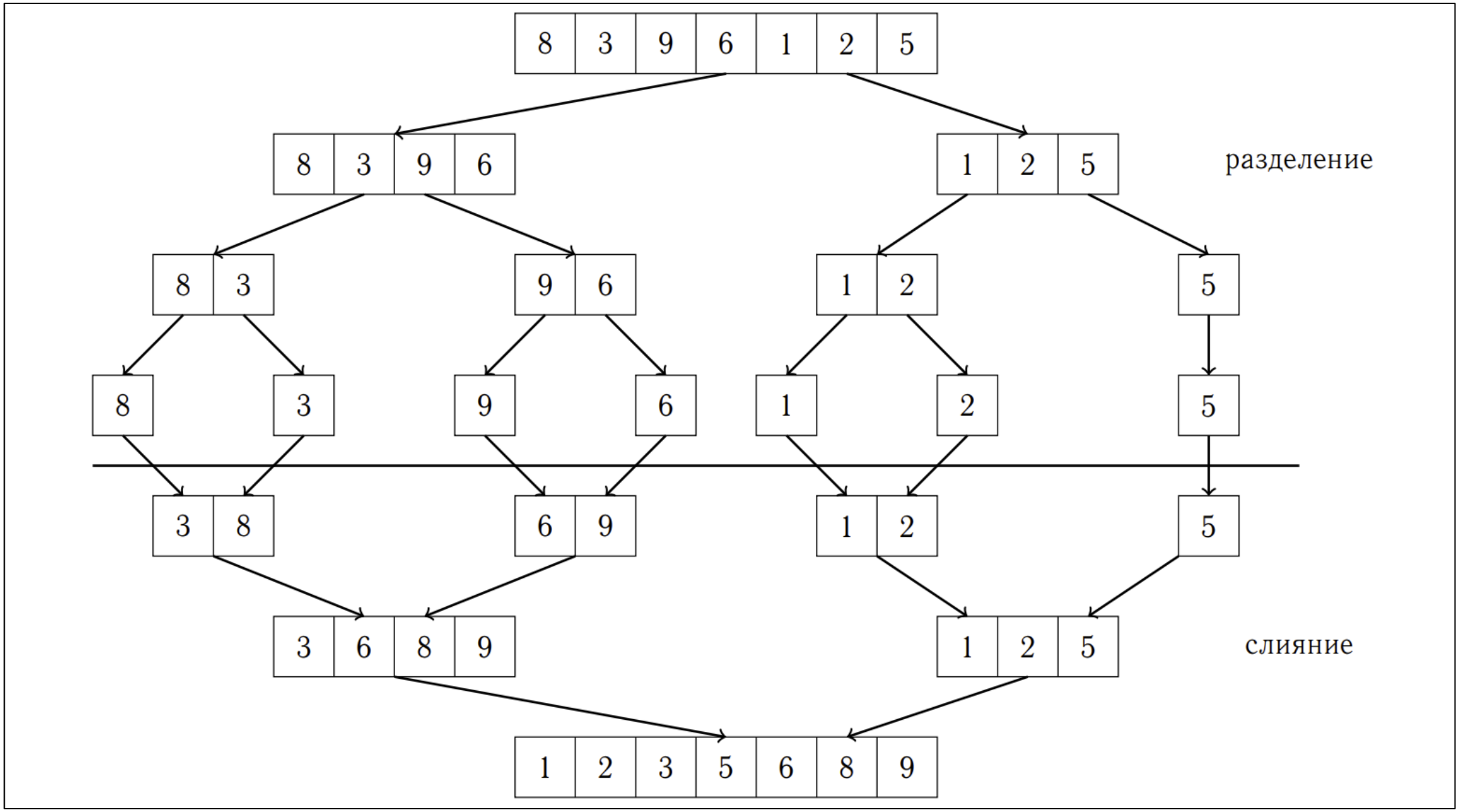
1. Сортируем массив данных.
2. Делим его пополам и находим середину.
3. Сравниваем срединный элемент с заданным искомым элементом.
4. Если искомое число больше среднего - продолжаем поиск в правой части массива (если он отсортирован по возрастанию): делим ее пополам, повторяя пункт 3. Если же заданное число меньше - алгоритм продолжит поиск в левой части массива, снова возвращаясь к пункту 3.

Рекуррентное соотношение: $T(n)=T(n/2)+O(1)$ => *сложность* $T(n)=O(\log n)$



Поиск 67 в отсортированном массиве

Пример: Сортировка слиянием



Преимущества метода

- Метод позволяет перейти от неэффективных обычных решений к более быстрым;
- Подзадачи независимы друг от друга;
- Сложные задачи, которые трудно решить целиком, становятся управляемыми при разбиении на простые подзадачи.

Спасибо за внимание!